

3534. Path Existence Queries in a Graph II

Solved

Hard Topics Companies Hint

You are given an integer `n` representing the number of nodes in a graph, labeled from 0 to `n - 1`.

You are also given an integer array `nums` of length `n` and an integer `maxDiff`.

An **undirected** edge exists between nodes `i` and `j` if the **absolute** difference between `nums[i]` and `nums[j]` is **at most** `maxDiff` (i.e., `|nums[i] - nums[j]| <= maxDiff`).

You are also given a 2D integer array `queries`. For each `queries[i] = [ui, vi]`, find the **minimum** distance between nodes `ui` and `vi`. If no path exists between the two nodes, return -1 for that query.

Return an array `answer`, where `answer[i]` is the result of the `ith` query.

Note: The edges between the nodes are unweighted.

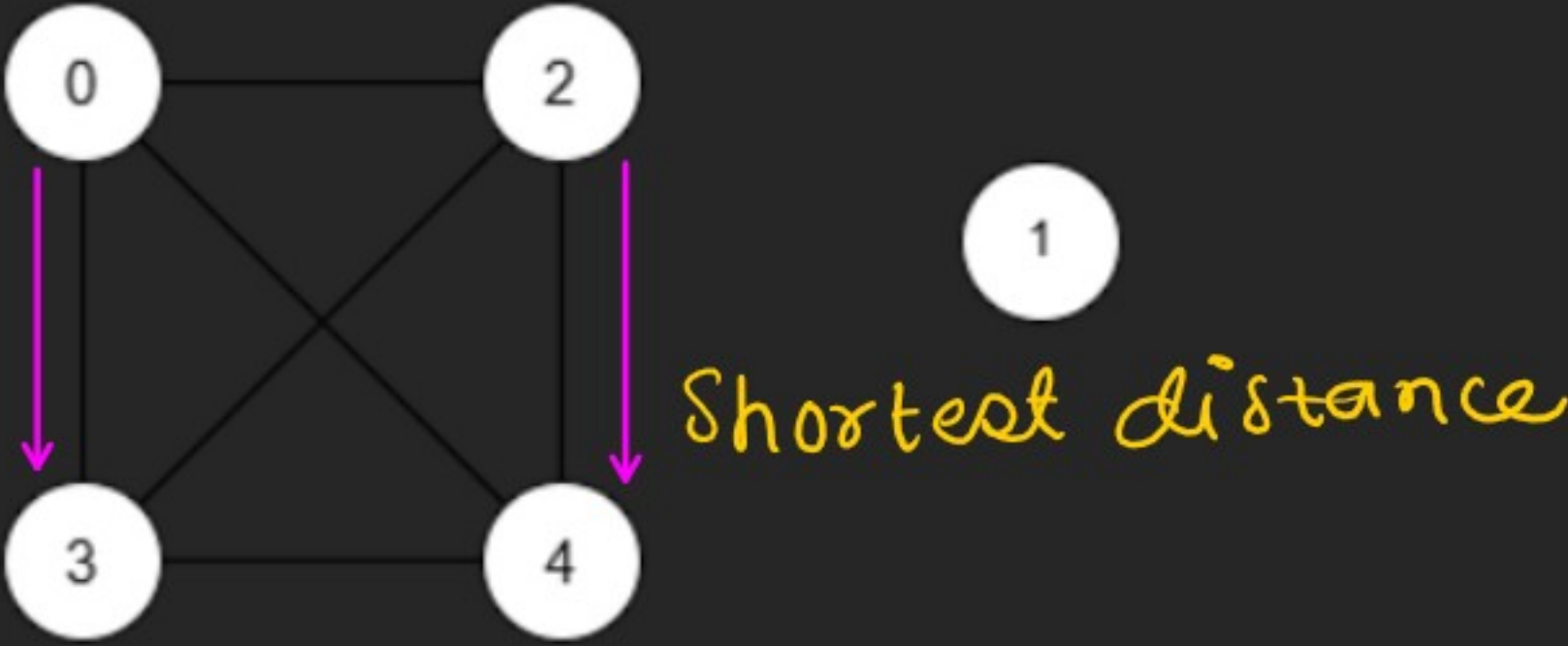
Example 1:

Input: `n = 5, nums = [1,8,3,4,2], maxDiff = 3, queries = [[0,3],[2,4]]`

Output: `[1,1]`

Explanation:

The resulting graph is:



Query	Shortest Path	Minimum Distance
[0, 3]	0 → 3	1
[2, 4]	2 → 4	1

Thus, the output is `[1, 1]`.

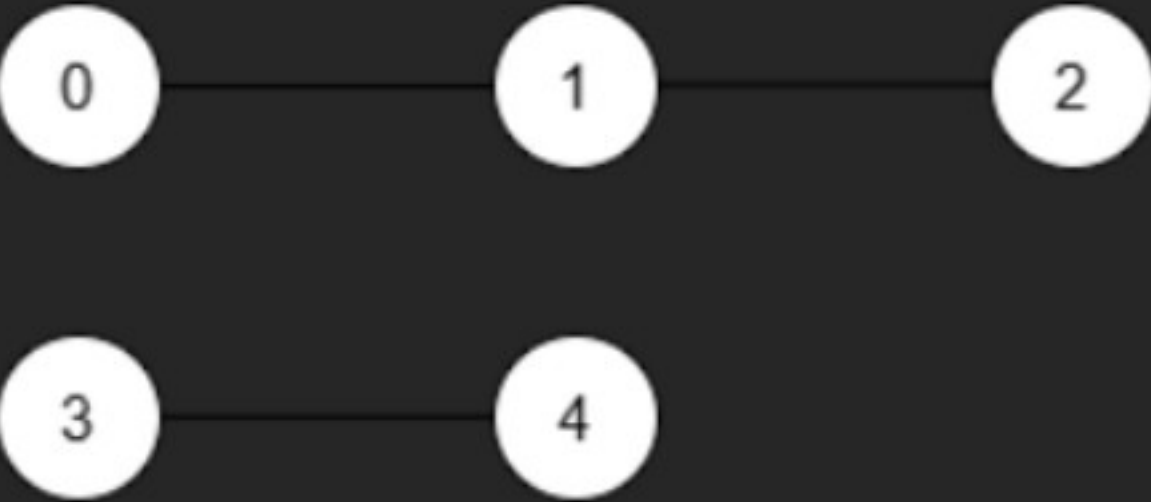
Example 2:

Input: `n = 5, nums = [5,3,1,9,10], maxDiff = 2, queries = [[0,1],[0,2],[2,3],[4,3]]`

Output: `[1,2,-1,1]`

Explanation:

The resulting graph is:



Query	Shortest Path	Minimum Distance
[0, 1]	0 → 1	1
[0, 2]	0 → 1 → 2	2
[2, 3]	None	-1
[4, 3]	3 → 4	1

Thus, the output is `[1, 2, -1, 1]`.

Example 3:

Input: `n = 3, nums = [3,6,1], maxDiff = 1, queries = [[0,0],[0,1],[1,2]]`

Output: `[0,-1,-1]`

Explanation:

There are no edges between any two nodes because:

- Nodes 0 and 1: `|nums[0] - nums[1]| = |3 - 6| = 3 > 1`
- Nodes 0 and 2: `|nums[0] - nums[2]| = |3 - 1| = 2 > 1`
- Nodes 1 and 2: `|nums[1] - nums[2]| = |6 - 1| = 5 > 1`

Thus, no node can reach any other node, and the output is `[0, -1, -1]`.

Constraints:

- `1 <= n == nums.length <= 105`
- `0 <= nums[i] <= 105`
- `0 <= maxDiff <= 105`
- `1 <= queries.length <= 105`
- `queries[i] == [ui, vi]`
- `0 <= ui, vi < n`

Thought Process

$$n=5$$

$$\text{nums} = \{ \overset{0}{5}, \overset{1}{3}, \overset{2}{1}, \overset{3}{9}, \overset{4}{10} \}, \text{maxDiff} = 2$$

$$\text{Queries} = \{(0,1), (0,2), (2,3), (4,3)\}$$

3 Brute Force

- BFS $\rightarrow Q$
- Dij $\rightarrow Q$
- Floyd $\rightarrow Q$

what if when sort it:

$$\text{arr} = \{(5,0), (3,1), (1,2), (9,3), (10,4)\}$$

$$\downarrow$$
$$\text{pair} \rightarrow \{\text{node}, \text{value}\}$$

Now sort it:

$$\text{arr} = \{ \overset{0}{(1,2)}, \overset{1}{(3,1)}, \overset{2}{(5,0)}, \overset{3}{(9,3)}, \overset{4}{(10,4)} \}$$

$\swarrow \quad \searrow$
value node



↑
not
connected

$$\text{nodeToIndex} = \begin{array}{|c|c|c|c|c|} \hline 0 & 1 & 2 & 3 & 4 \\ \hline 2 & 1 & 0 & 3 & 4 \\ \hline \end{array}$$

$$\text{Query}(0,2)$$

$\uparrow \quad \uparrow$
u v

$$a = \text{nodeToIndex}[u]$$

$$b = \text{nodeToIndex}[v]$$

$$\text{arr} = \{ \overset{0}{(1,2)}, \overset{1}{(3,1)}, \overset{2}{(5,0)}, \overset{3}{(9,3)}, \overset{4}{(10,4)} \}$$

$\swarrow \quad \searrow$
value node

↖ a jumping ↗
↖ a ↗



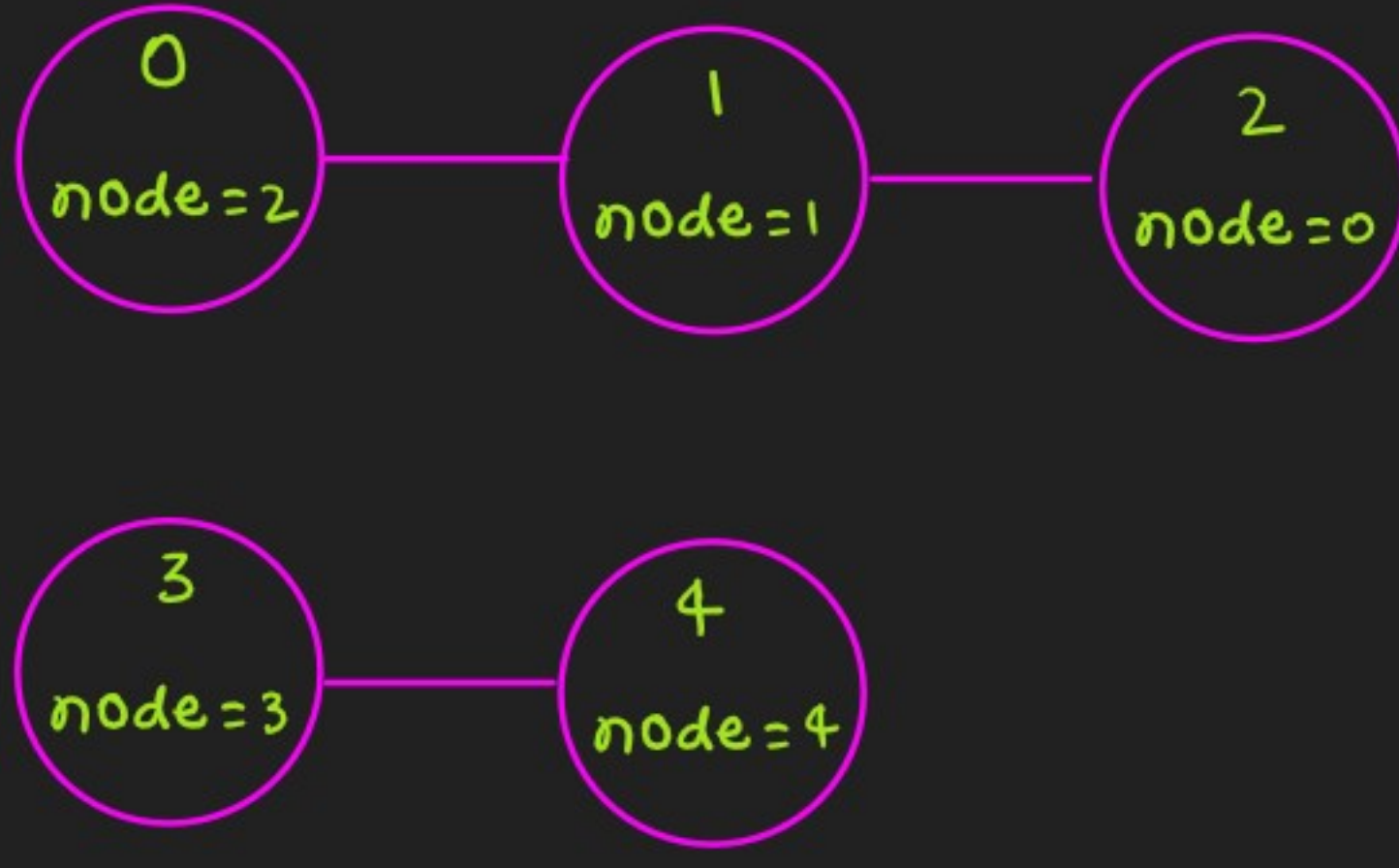
$$a = \text{nodeToIndex}[u] = \cancel{2} 0$$

$$b = \text{nodeToIndex}[v] = \cancel{0} 2$$

$$\text{if}(a > b) \text{ swap}(a, b)$$

Jumping Efficiently →

Binary lifting



- $\gamma_{OW} S = \sigma$

$2^0 \rightarrow \text{Jump}$
 $2^1 \rightarrow \text{Jump}$
 $j^0 = 0$ $j^1 = 1$ $j^2 = 2$

node 0	1		
node 1	2		
node 2	2		
node 3	4		
node 4	4		

$$\begin{aligned} \text{cols} &= \log_2(n) + 1 \\ &= \log_2(8) + 1 \\ &= 3 \end{aligned}$$

- fastest node s can reach:

$$\text{arr} = \{(\overset{0}{1}, \overset{1}{2}), (\overset{2}{3}, \overset{3}{1}), (\overset{4}{5}, \overset{5}{0}), (\overset{6}{9}, \overset{7}{3}), (\overset{8}{10}, \overset{9}{4})\}$$

$$\text{arr} = \{(1, 2), (3, 1), (5, 0), (9, 3), (10, 4)\}$$

↓

$$\text{value} = 3 + \text{maxDiff}$$

$$= 3 + 2$$

$$= 5$$

$$\text{arr} = \{(1, 2), (3, 1), (5, 0), (9, 3), (10, 4)\}$$

↓

$$\text{value} = s + \text{maxDiff}$$

$$= 5 + 2$$

$$= 7$$

$$\text{arr} = \{(1, 2), (3, 1), (5, 0), (9, 3), (10, 4)\}$$

↓

$$\text{value} = 9 + \text{maxDiff}$$

$$= 9 + 2$$

$$= 11$$

- we find column 0 first

```
for (node = 0; node < n; node++) {  
    int farthestNodeOneHop =  
        customBinary = (arr, arr[node].  
                        first + maxDiff);  
}
```

// fill remaining column (Binary diffing)

```

for (j=1; j<cols; j++) {
    for (node=0; node<n; node++) {
        ancestorTable[node][j] = ancestor
            Table[ancestorTable[node][j-1]]
                [j-1];
    }
}

```


Let's jump efficiently

	$j=0$	$j=1$	$j=2$
node 0	1	2	2
node 1	2	2	2
node 2	2	2	2
node 3	4	4	4
node 4	4	4	4

ancestor table

query(0, 2)

$a = \text{nodeToIndex}[0]; // 2$

$b = \text{nodeToIndex}[2]; // 0$

$a = 2$

$b = 0$

if ($a > b$) swap(a, b);

Now,

$a = 0$

$b = 2$

$\text{curr} = a; // 0$

$\text{jump} = 0;$

$j = 2 \rightarrow 2^2 \text{ jump}$

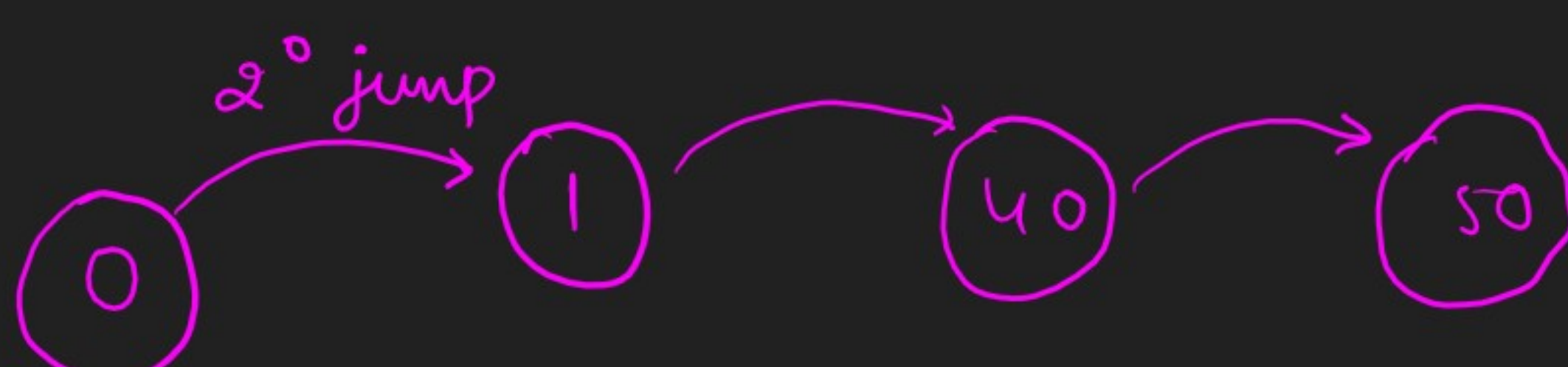
$\text{ancestorTable}[0][2] = 2 < b$
 $\downarrow$
 move next

$j = 1 \rightarrow 2^1 \text{ jump}$

$\text{ancestorTable}[0][1] = 2 < b$

$j = 0 \rightarrow 2^0 \text{ jump}$

$\text{ancestorTable}[0][0] = 1 < b$



$\text{curr} = 1$

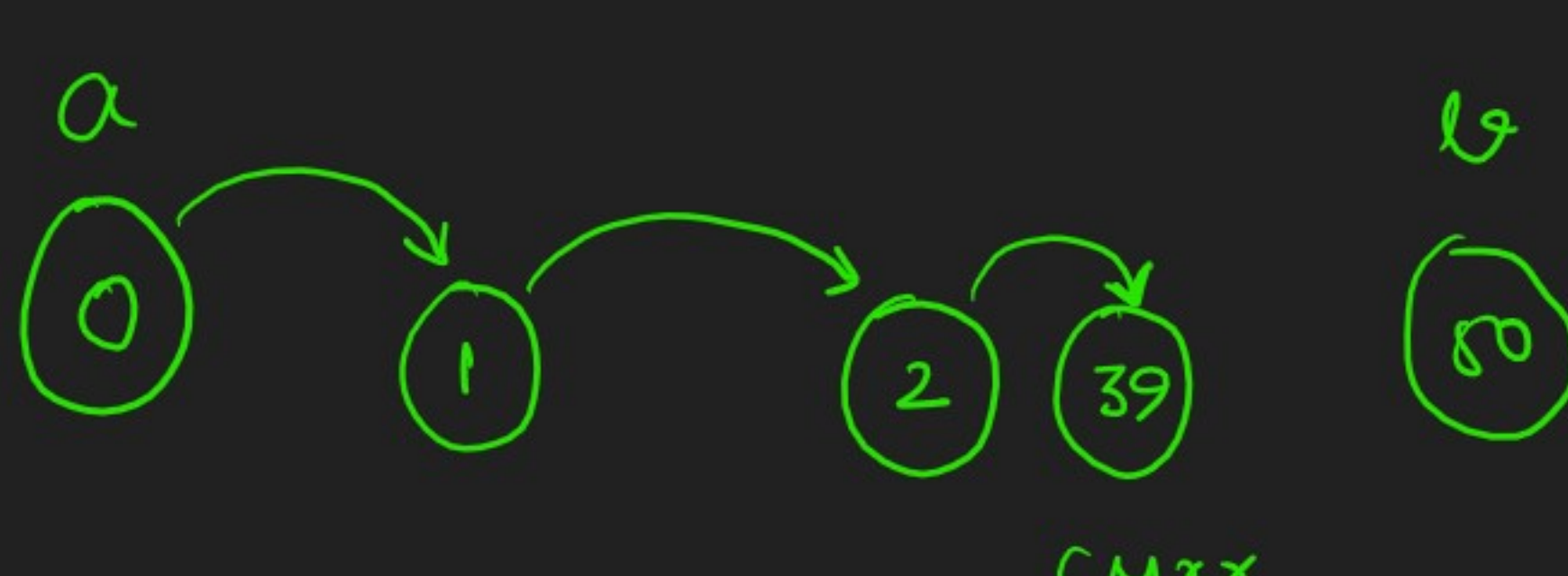
$\text{jump} = 2^0 = 1$

for ($j = \text{col} - 1; j \geq 0; j--$) {

if ($\text{ancestor}[\text{curr}][j] < b$) {

$\text{curr} = \text{ancestor}[\text{curr}][j]$

$\text{jump} += (1 < j); // 2^j$



- curr is standing at the farthest possible node which is $< b$

if ($\underbrace{\text{ancestorTable}[\text{curr}][0]}_{\text{node}} \geq b$) {

$\text{result.push_back}(\text{jump} + 1)$

else {

$\text{result.push_back}(-1);$